Conception et implémentation d'une application multitâche



Nous ne cherchons pas à faire dans ce projet une application temps réel. Les APIs que vous allez manipuler, les phénomènes que vous allez observer et la conception que vous allez mettre en œuvre dans ce projet seraient reproductibles sur un système d'exploitation dit «Temps Réel».

Objectifs

Les objectifs de ce projet sont multiples :

- Concevoir des applications multitâche (non temps réel) avec une approche dirigée par les événements
- Manipuler les mécanismes POSIX d'un système d'exploitation Linux
- Manipuler les mécanismes C11
- Concevoir une application multitâche avec une approche dirigée par le temps

Consignes

Vous devez rendre un compte rendu rédigé en l'intégrant dans une archive .tar.gz accompagnée de vos codes sources et uniquement de vos codes sources. Les binaires seront au préalable supprimés en utilisant la commande `make clean`.

Ce projet archivé sera à envoyer par email à l'adresse `fthomas.fr@gmail.com` à la fin de la deuxième séance. A la fin de cette deuxième séance, vous devrez venir vérifier auprès de l'enseignant la reception de votre projet et signer la feuille d'émargement.

L'ensemble des informations et codes d'exemples concernant la programmation multitâche sont disponibles dans votre cours.

Sources

Le code source pour ce projet est présent à l'adresse suivante :

https://github.com/fthomasfr/multitasking_training_practical_work

Pour le récupérer vous pouvez le télécharger directement ou utiliser git :

```
git clone https://github.com/fthomasfr/multitasking_training_practical_work.git
```

Préambule

L'objectif de ce préambule est de vous faire découvrir la création d'une tâche et d'un sémaphore en utilisant les APIs POSIX. Ces APIs seront utilisées dans l'exercice principal.

Pour compiler le programme de ce preambule, un makefile est proposé avec les règles suivantes:

- `make preambule`: compile le programme de préambile incluant le fichier `preambule.c`
- `make runpreambule`: compile et execute le programme de préambile incluant le fichier `preambule.c`
- `make clean`: supprime les executables et l'ensemble des fichiers et artefacts de compilation
- `make pumls`: génère les images png à partir des descriptions plantuml stockées dans le répertoire `diagrams`

1. Ouvrez le fichier `preambule.c` avec l'éditeur de votre choix.



Pour visualiser la documentation d'une API POSIX man [nom de la méthode] dans une ligne de commande

Copyright (c) 2017-2026 Frédéric Thomas

A l'aide de la partie 3 de votre cours présentant les APIs POSIX:

2. Identifiez dans le programe le nom du thread.
3. Identifiez dans le programe le nom du sémaphore.
4. Identifiez dans le programe le nom du mutex.
5. Identifiez la création du thread.
6. Identifiez dans le programe le point d'entré du thread.
7. Identifiez l'attente de la fin du thread pour terminer le processus courant.
8. Identifiez le thread par défaut du processus courant.
9. Identifiez les processus que vous allez mettre en jeu pour exécuter votre programme.
10. Nous allons utiliser le logiciel plantuml pour réaliser de manière automatique nos diagrammes de séquences UML (L'objectif de ce TP n'est pas de vous apprendre l'UML). Complétez le diagramme ci-dessous expliquant la conception détaillée de ce processus et de ce thread en remplaçant les `[TODO]`. Vous devez exporter votre diagramme pour l'intégrer à votre compte rendu. Seul le diagramme dans le compte rendu compte.

Deux solutions pour générer le diagramme: un éditeur plantuml est utilisable directement en ligne plantText ou l'utilisation du container `Docker plantuml` en local sur votre machine.

- Si vous optez pour l'éditeur en ligne: copiez/collez le code présent dans le fichier `diagrams/multitaskingAccumulatorClassDiagram.puml` dans l'éditeur en ligne puis cliquez sur `Refresh`.
- Si vous optez pour le container, `make pumls` doit vous donner le diagramme ci-dessous.

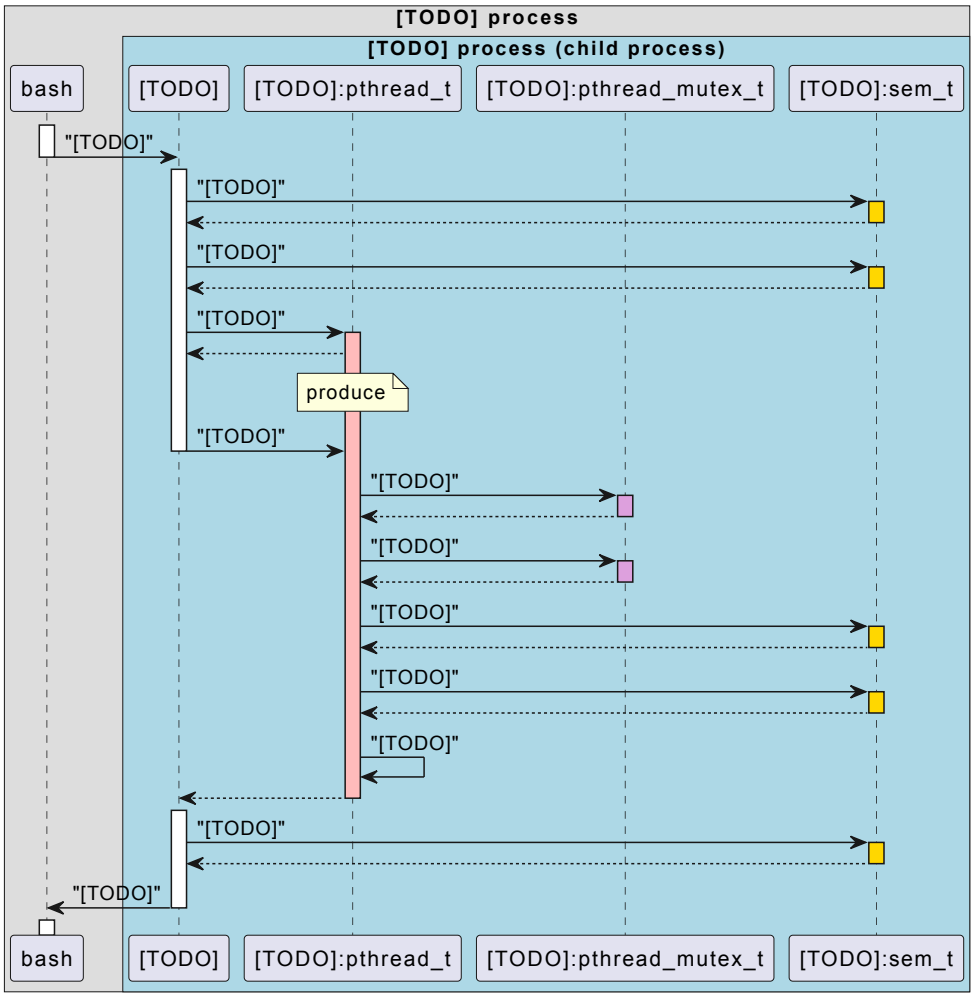


Figure 1: Conception détaillée du processus de découverte des APIs POSIX. La description de l'architecture dynamique du processus de découverte des APIs POSIX.

11. En partant de l'hypothèse qu'il n'existe pas d'exigences temporelles sur ce programme, sa conception est-elle complète ?

Exercice-1: MultitaskingAccumulator

Copyright (c) 2017-2026 Frédéric Thomas

Comme le montre le schéma d'architecture système de la figure 2, nous cherchons à développer un accumulateur nommé `MultitaskingAccumulator` qui réalise l'acquisition de quatre entrées numériques asynchrones auprès d'un composant logiciel externe nommé `SensorManager` et qui somme ces entrées. Il produit une sortie numérique correspondante à cette somme et une sortie de diagnostic. Ces sorties sont connectées à un `Display` pour affichage.

Le `SensorManager` met à disposition pour chaque entrée un tableau de 256 entiers et un checksum modélisés par le type de donnée `MSG_BLOCK`. La sortie numérique est caractérisée par un tableau de 256 entiers et un checksum modélisés par le même type de donnée `MSG_BLOCK`. La sortie diagnostic est une chaine de caractères à destination du terminal.

La fonction `getInput` de `SensorManager` permet de simuler la production d'une entrée. La fonction `messageDisplay` de `Display` permet d'afficher le message. De même, la fonction `print` permet d'afficher le nombre de messages produits, consommés et la différence entre les deux.

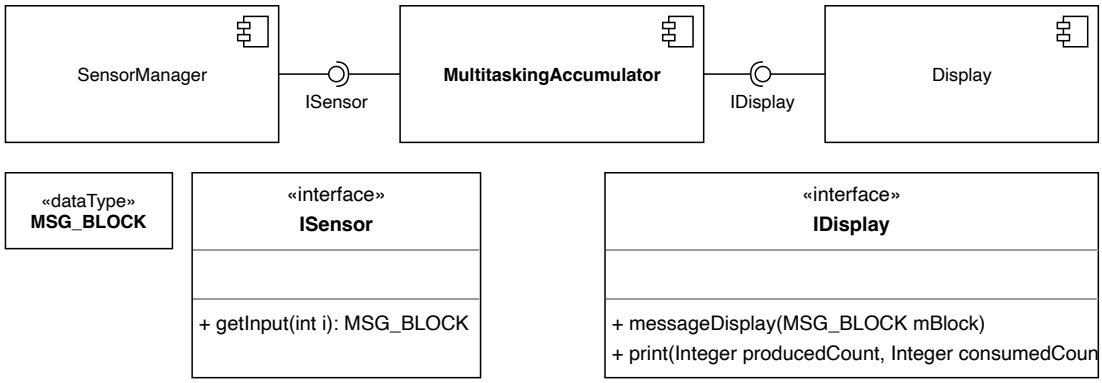


Figure 2: Architecture Système. La description de l'architecture système dans laquelle notre logiciel est intégré.

Depuis cette architecture système les exigences suivantes ont été assignées à `MultitaskingAccumulator` :

- **Exigence 1** : MultitaskingAccumulator doit produire sur sa sortie `IDisplay` (`messageDisplay`) le cumul de quatre `ISensor` dès qu'une entrée est acquise sur `ISensor`. Le cumul correspond à la production d'un tableau de 256 entiers dont le ieme élément de ce tableau est la somme des iemes éléments des tableaux d'entrée.
- **Exigence 2** : MultitaskingAccumulator doit acquérir toutes les données fournies sur son entrée `ISensor`.
- **Exigence 3** : MultitaskingAccumulator doit garantir que les données d'entrée de `ISensor` sont correctement formées avant de les sommer pour produire le cumul sur `IDisplay`, sinon il doit supprimer la donnée acquise et relever un message d'erreur dans ses logs produits sur sa sortie standard.
- **Exigence 4** : MultitaskingAccumulator doit sommer et produire une sortie sur `IDisplay` au plus vite, c'est-à-dire dès qu'une entrée est présente sur `ISensor` sans attendre une nouvelle valeur sur chacune des entrées.
- **Exigence 5** : MultitaskingAccumulator doit produire un diagnostic sur sa sortie `IDisplay` (`print`) de manière asynchrone vis à vis du cumul. Ce diagnostic explicite combien d'entrées ont été acquises, combien ont été sommées et combien restent à sommer. Ces métriques doivent être cohérentes avec le cumul produit sur `IDisplay` (`messageDisplay`). Il n'est pas exigé qu'un diagnostic soit généré pour chaque cumul.

Note: Pour l'exigence 1, nous cherchons à développer un accumulateur. Il est donc inutile d'attendre les quatres entrées pour produire une somme. Au fur et à mesure des valeurs en entrée, le logiciel doit sommer et cela indépendamment de l'entrée. Nous devons sommer deux valeurs successives de la première entrée si elles arrivent avant une valeur sur la deuxième entrée par exemple.

Un architecte logiciel a décrit dans un Software Architecture Document (SAD), l'architecture de `MultitaskingAccumulator` à mettre en œuvre :

MASTER

Copyright (c) 2017-2026 Frédéric Thomas

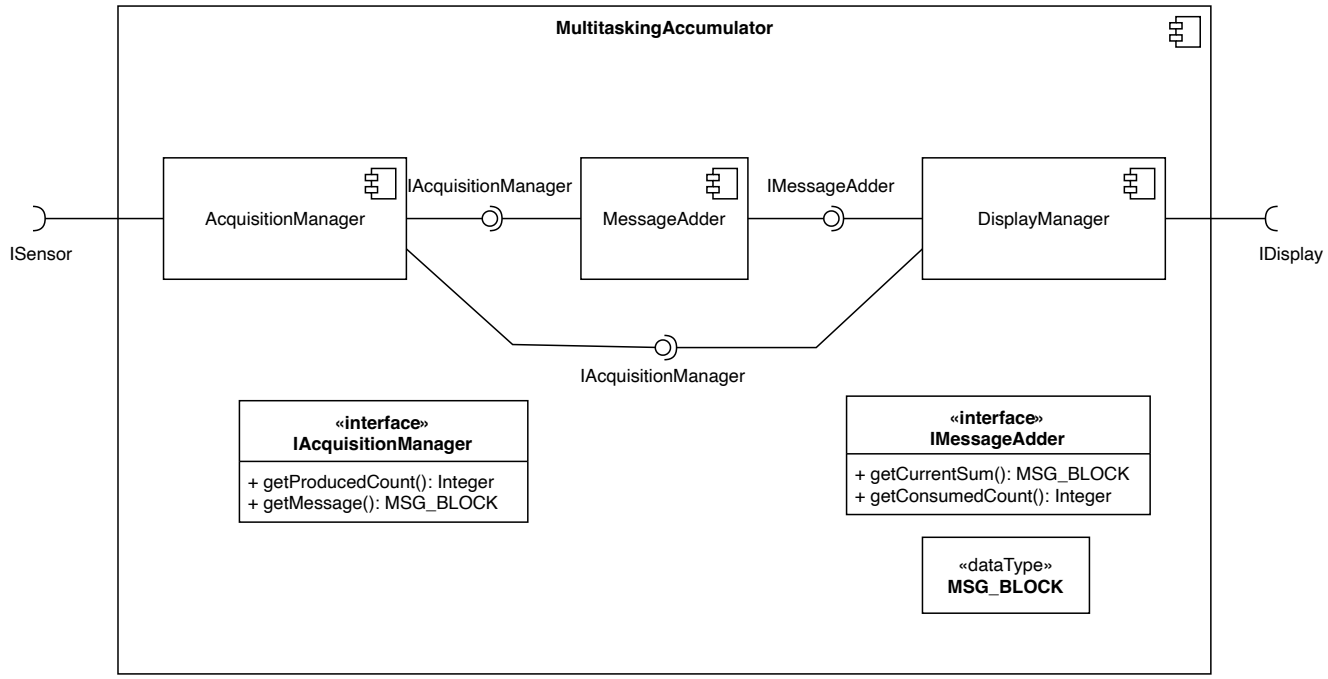


Figure 3: Architecture Logicielle de MultitaskingAccumulator. La description de l'architecture logicielle à mettre en oeuvre.

`AcquisitionManager` acquiert les entrées depuis l'interface `ISensor`. `MessageAdder` somme ces entrées. `DisplayManager` pilote la sortie. `MessageAdder` obtient un nouveau message produit par la méthode `getMessage` d'`AcquisitionManager`. `MessageAdder` et `DisplayManager` peuvent obtenir le nombre de messages produits par la méthode `getProducedMessage`. De même, `DisplayManager` obtient le nombre de messages consommés et la somme courante par les méthodes `getConsumedCount` et `getCurrentSum` fournies par `MessageAdder`.

1. Complétez cette architecture logicielle en allouant les exigences de la spécification précédente sur les composants de `MultitaskingAccumulator`. Un exemple d'allocation à compléter et à modifier est illustré en figure 4. Pour cela, les sources de ce diagramme à compléter sont rangées dans le fichier `diagrams/conception.drawio` editable avec le programme en ligne draw.io (Importez ce fichier dans l'éditeur en ligne pour gagner du temps). **(0,5 POINT)**

Copyright (c) 2017-2026 Frédéric Thomas

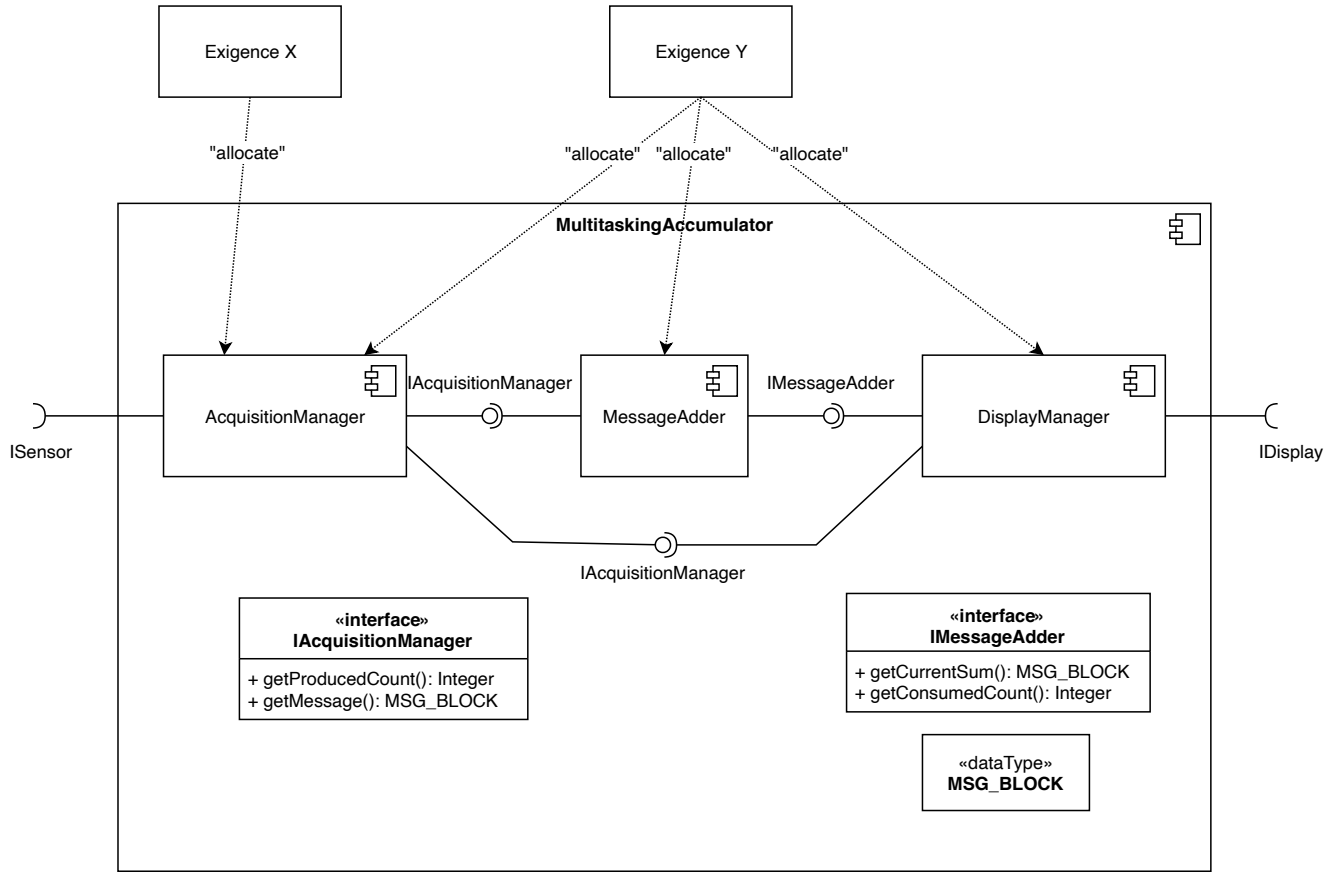


Figure 4: Architecture Logicielle avec allocation des exigences. Un exemple d'allocation d'exigence sur les composants logiciels à modifier/compléter.

L'objectif de ce projet est de décrire l'architecture détaillée multitâche et de proposer plusieurs implémentations.

Conception Détaillée en utilisant une approche dirigée par les événements

2. Une approche dirigée par les événements est-elle une approche synchrone ou asynchrone ? **(0,5 POINT)**

Plusieurs conceptions détaillées peuvent exister. Nous proposons de guider votre conception en suivant les choix de conception suivants:

- Utilisation de tâches où la mémoire est partagée entre les tâches;
- Appel de la fonction `getInput` dans quatres tâches différentes (une pour chaque entrée) pour aller cherche les données dans `ISensor`. Il n'y a pas d'événement dans `ISensor` indiquant la présence d'une nouvelle entrée. Dès qu'elle est prête la tâche appel `getInput`.
- Nous n'attendons pas toutes les entrées pour sommer, dès qu'une entrée est disponible, elle est le mise dans un tampon en attente d'être sommée. `MessageAdder` cumule cette entrée avec la somme précédente.
- Partage d'un seul tampon de messages `buffer` de taille fixe (taille que vous devez definir) pour échanger les données entre `AcquisitionManager` et `MessageAdder`.
- Nous réduisons au maximum les sections critiques pour fluidifier le programme. Plusieurs écrivains doivent pouvoir écrire en parrallèle dans le tampon typiquement.
- Utilisation d'événements sans transmission de données pour synchroniser les tâches;
- Utilisation d'un checksum pour satisfaire l'exigence 3;
- L'utilisation des librairies de messages simulant les entrées (`SensorManager`) et l'affichage (`Display`). La fonction `getInput` permet de simuler la production d'une entrée. La fonction `messageDisplay` permet d'afficher le message. La fonction `print` permet d'afficher le nombre de messages produits, consommés et la différence entre les deux.
- L'utilisation du module `msg.h`/`msg.c` qui fournit des fonctions utilitaires pour sommer deux messages et vérifier le checksum d'un message.

Nous proposons une architecture détaillée préliminaire en figure 5 et un squelette d'implémentation de cette architecture dans les `.h` et `.c` fournis. Etudiez le squelette d'implémentation `.h` et `.c` fournis et l'architecture logicielle détaillée ci-dessous pour en comprendre les structures de données et les APIs.

Copyright (c) 2017-2026 Frédéric Thomas

Pour ce diagram d'architecture détaillée nous avons utilisé le logiciel plantuml. Deux solutions: un éditeur plantuml est utilisable directement en ligne plantText ou l'utilisation du container `Docker plantuml` en local sur votre machine.

- Si vous optez pour l'éditeur en ligne: copiez/collez le code présent dans le fichier `diagrams/multitaskingAccumulatorClassDiagram.puml` dans l'éditeur en ligne puis cliquez sur `Refresh`.
- Si vous optez pour le container, `make pumls` doit vous donner le diagramme ci-dessous.

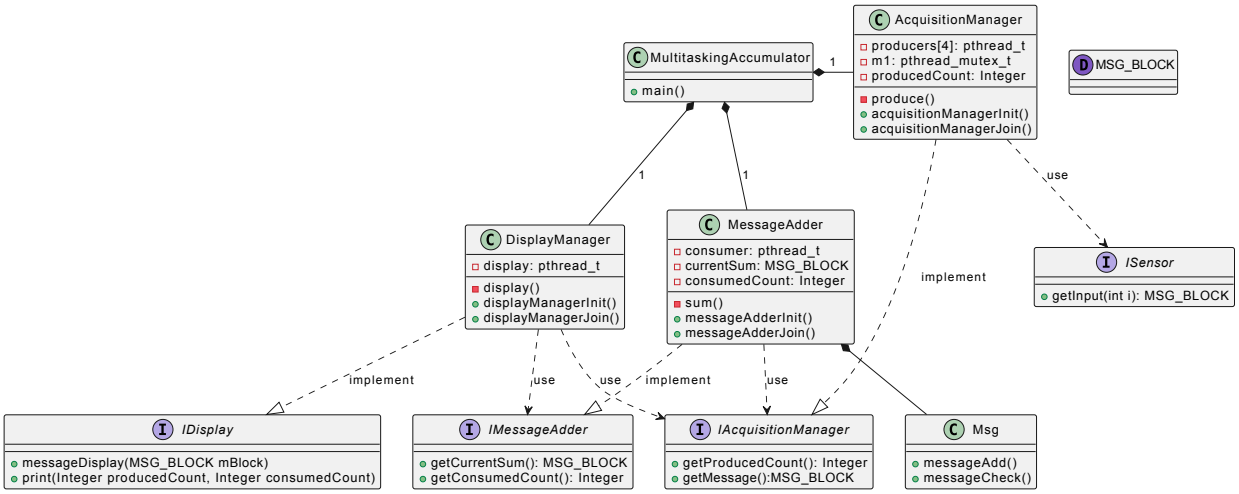


Figure 5: Architecture Logicielle Détaillée de MultitaskingAccumulator. La description de l'architecture logicielle détaillée mise en oeuvre.

3. A quelle stratégie de sûreté la méthode messageCheck répond-t-elle ? **(1 POINT)**
4. A ce stade de l'implémentation squelette proposée, combien y a-t'il de processus (`process`) et de fil d'exécution (`thread`) POSIX dans ce programme ? **(0,5 POINT)**
5. Comment est calculé le checksum d'un message dans la fonction `messageCheck` ? **(1 POINT)**
6. Complétez cette architecture logicielle détaillée par l'analyse des tâches à mettre en oeuvre, des échanges de données et des synchronisations entre les tâches en satisfaisant les choix de conception précédents. **(3 POINTS)**

Cette conception devra donc détailler l'architecture statique et dynamique. Vous devrez mettre à jour le diagramme de classes pour y ajouter les APIs dont vous avez besoin et que vous utilisez dans le diagramme de séquence. Vous devrez fournir dans votre rapport ce diagramme de classes et un diagramme de séquence.

Le diagramme de séquences est utilisé pour expliquer et démontrer la causalité des traitements. Un exemple de diagramme de séquence est illustré ci-dessous. Vous pouvez repartir de cet exemple en utilisant le formalisme plantuml disponible en ligne pour réaliser de manière automatique vos diagrammes de séquences UML. Cela doit vous donner le diagramme ci-dessous. Le diagramme de classes est soit réalisés à partir de zéro dans plantText soit complété à partir du fichier drawio disponible dans le répertoire `diagrams/conception.drawio`.

Vous indiquerez dans votre conception les valeurs initiales des sémaphores si vous utilisez des sémaphores.

Pour rappel, nous souhaitons suivre les bonnes pratiques de conception notamment celle "Acquire and release synchronization primitives in the same module, at the same level of abstraction" vue en cours. Si vous avez besoins de sémaphore et mutex par exemple, cela implique de créer des accesseurs pour limiter l'utilisation de ces sémaphores et mutex au seul module C qui les déclare (voir `getMessage` dans la figure 5). Il vous faudra à minima les fonctions `incrementProducerCount` et `getProducerCount` dans l'`AcquisitionManager` pour la suite du projet. La fonction `incrementProducerCount` pourra rester locale au module. `getProducerCount` devra être publique.

MASTER

Copyright (c) 2017-2026 Frédéric Thomas

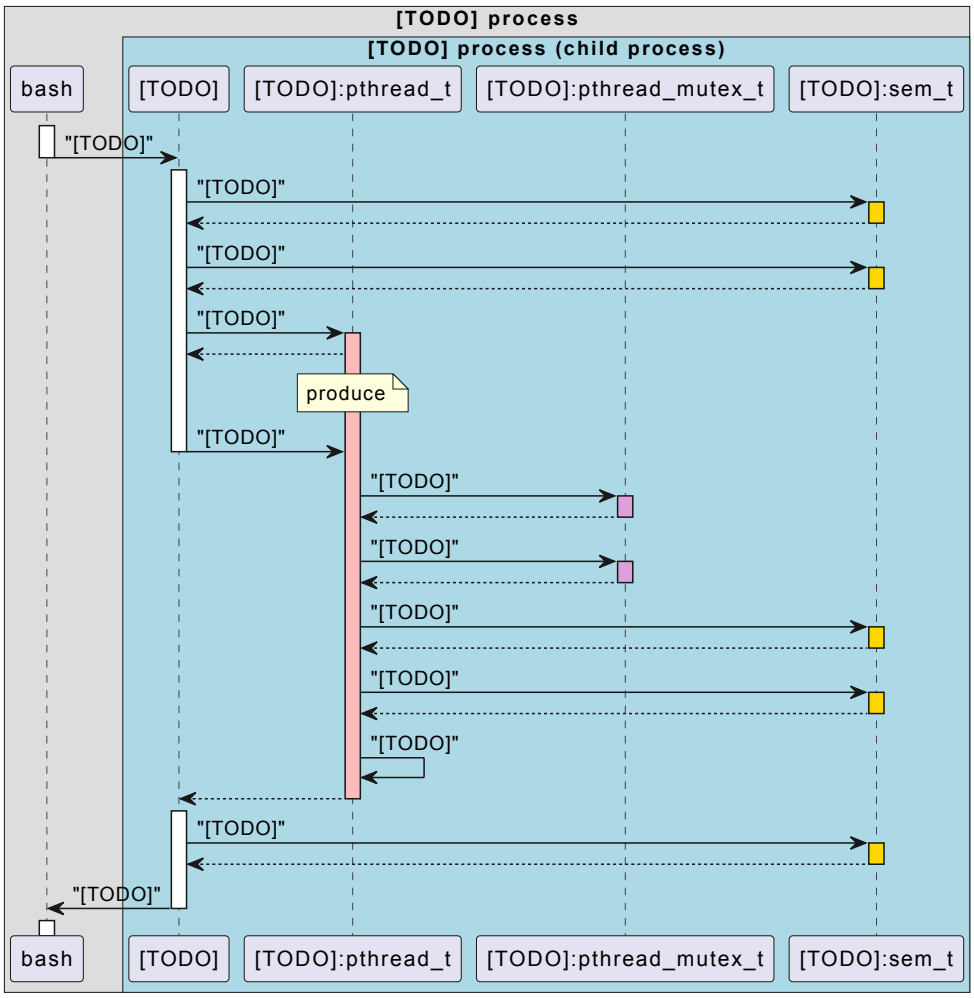


Figure 6: Conception détaillée pour illustrer le comportement dynamique de MultitaskingAccumulator. Un exemple de description du comportement dynamique à compléter/modifier.

Implémentation dirigée par les événements

7. Implémentez votre conception (Implémentation + Exécution) et montrez un résultat d'exécution. **(3 POINTS)**

   Pour debugger, il est possible de mettre une valeur fixe lors de la création du message en ligne 13 du fichier `sensorManager.c`.
8. Expliquez comment vous avez rendu cohérent la sortie diagnostic `IDisplay` (`print`) et le cumul produit sur `IDisplay` (`messageDisplay`) ? Nous ne voulons pas, bien entendu, une incohérence à un instant donné entre la somme reçue par `DisplayManager` et le nombre de message consommé connu de `DisplayManager`. L'ensemble doit être cohérent à chaque affichage. **(1 POINT)**
9. Pourquoi certaines variables sont-elles considérées comme des variables C volatiles dans l'implémentation proposée ? **(1 POINT)**

Copyright (c) 2017-2026 Frédéric Thomas